

✓ Part 1: Implementing Convolutions in PyTorch

In this lab, you will learn the fundamentals of convolution operations, which are a crucial building block in convolutional neural networks (CNNs). We will start by understanding 1D convolutions, implementing one as a warm-up. You will then move on to implement 2D convolutions, preparing you for building a custom CNN.

✓ Part 1.1: Implementing a 1D Convolution

Below is the provided code for a 1D convolution implementation in PyTorch. Review this code and ensure you understand each step.

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
def conv_1d(x, wt, b, p=0):
    """ A naive implementation of 1D convolution in PyTorch.

    Input:
    - x: a 1D tensor of shape (n,)
    - wt: a 1D tensor of shape (f,)
    - b: a scalar bias tensor
    - p: an integer padding size

    Returns:
    - out: a 1D tensor of shape (m,) where m = n + 2 * p - f + 1
    """

    # Ensure all inputs are on the same device
    device = x.device
    wt = wt.to(device)
    b = b.to(device)

    # Calculate the shape of the output
    f = wt.shape[0]
    n = x.shape[0]
    m = n + 2 * p - f + 1

    out = torch.zeros(m, dtype=x.dtype, device=x.device)

    # Pad the input
    if p > 0:
        x_padded = F.pad(x, (p, p), mode='constant')
    else:
        x_padded = x

    # Perform the convolution
    for i in range(m):
        out[i] = torch.sum(x_padded[i:i+f] * wt) + b

    # Return the output
    return out
```

```
requires_grad = False

x = torch.tensor([0, 0, 1, 1, 1, 0, 1, 0, 0, 0], dtype=torch.float32, requires_grad=requires_grad)
wt = torch.tensor([-1, 1, -1], dtype=torch.float32, requires_grad=requires_grad)
b = torch.tensor(1, dtype=torch.float32, requires_grad=requires_grad)
p = 1

out_expected = torch.tensor([1, 0, 1, 0, 1, -1, 2, 0, 1, 1], dtype=torch.float32)

out = conv_1d(x, wt, b, p)
print("output of conv_1d is:\n", out)

# compare the output with the expected output
```

```
result = torch.allclose(out, out_expected, atol=1e-4)
print("Is the result correct? ", result)
```

```
output of conv_1d is:
tensor([ 1.,  0.,  1.,  0.,  1., -1.,  2.,  0.,  1.,  1.])
Is the result correct? True
```

Part 1.2: Implementing 2D Convolution

Now that you've seen a 1D convolution, let's extend this concept to 2D data. This is a key step, as most image data is represented in 2D or 3D (with multiple channels).

Question: Implement the `conv_2d` function, a basic 2D convolution function, following the structure of the provided 1D convolution. This function should accept a 2D input tensor `x`, a 2D filter `wt`, and a bias value `b`, and should apply zero-padding with size `p` to `x`.

```
def conv_2d(x, wt, b, p=0):
    """ A naive implementation of 2D convolution in PyTorch.

    Input:
    - x: a 2D tensor of shape (h, w)
    - wt: a 2D tensor of shape (f, f)
    - b: a scalar bias tensor
    - p: an integer padding size

    Returns:
    - out: a 2D tensor of shape (h', w') where
        h' = h + 2 * p - f + 1
        w' = w + 2 * p - f + 1
    """

    # Ensure all inputs are on the same device
    device = x.device
    wt = wt.to(device)
    b = b.to(device)

    # Calculate the shape of the output
    h, w = x.shape
    f = wt.shape[0] # assuming square filter f x f

    h_out = h + 2 * p - f + 1
    w_out = w + 2 * p - f + 1

    out = torch.zeros((h_out, w_out), dtype=x.dtype, device=x.device)

    # Pad the input
    if p > 0:
        # F.pad for 2D expects (pad_left, pad_right, pad_top, pad_bottom)
        x_padded = F.pad(x, (p, p, p, p), mode='constant')
    else:
        x_padded = x

    # Perform the convolution
    for i in range(h_out):
        for j in range(w_out):
            out[i, j] = torch.sum(x_padded[i:i+f, j:j+f] * wt) + b

    # Return the output
    return out
```

Question:

Complete the testing code below by writing the expected output. Then, verify the correctness of your implementation of function `conv_2d`.

If your implementation is not correct, try to debug your code by printing out the values and shapes of the intermediate results in your function.

```
requires_grad = False

x = torch.tensor([[1, 0, 1, 0, 0],
                  [1, 0, 1, 0, 1],
                  [1, 1, 1, 0, 0],
                  [1, 0, 1, 0, 1],
                  [1, 0, 1, 0, 1]])
```

```

dtype=torch.float32,
requires_grad=requires_grad)

wt = torch.tensor([[-1, -1, -1],
                  [-1, 1, -1],
                  [-1, -1, -1]],
                  dtype=torch.float32,
                  requires_grad=requires_grad)

b = torch.tensor(0, dtype=torch.float32, requires_grad=requires_grad)

p = 1

out_expected = torch.tensor([[-1., -5., -1., -2., 0.],
                             [-2., -7., -4., -1., -2.],
                             [-2., -5., -3., -2., -2.],
                             [-2., -7., -3., -3., -2.],
                             [ 0., -4., -1., 0., -1.]], dtype=torch.float32)

out = conv_2d(x, wt, b, p)
print("output of conv_2d is:\n", out)

# compare the output with the expected output
result = torch.allclose(out, out_expected, atol=1e-4)
print("Is the result correct? ", result)

```

```

output of conv_2d is:
tensor([[ 0., -4.,  0., -3., -1.],
        [-2., -7., -2., -4.,  1.],
        [-2., -5., -2., -5., -2.],
        [-2., -7., -2., -5.,  0.],
        [ 0., -4.,  0., -4.,  0.]])
Is the result correct? False

```

✓ Part 2: CNN on FashionMNIST Dataset

✓ Part 2.1: Implementing Batch 2D Convolution

In real-world applications, we often work with batches of images and multiple filters. The next step is to implement `conv_2d_batch`, a batch convolution function that applies several filters to a batch of images.

When training neural networks, especially on large datasets, we often process images in batches instead of one at a time. This helps speed up training and is efficient for computational hardware like GPUs. To understand batch 2D convolution, let's break down the main elements it involves:

1. Batch Size:

- A batch is a collection of images (or other data) processed simultaneously. For example, if our batch size is 64, we process 64 images at once, instead of just one image.
- Batch processing enables parallel computation, which is faster and more efficient, especially on GPUs.
- In your tensor, the batch dimension is the first dimension, denoted as n (the number of images in the batch).

2. Channels:

- Channels represent the different layers of information in an image. A grayscale image has 1 channel, while an RGB (color) image has 3 channels (Red, Green, Blue).
- When performing convolution on an image, we apply the filter across each channel and sum the results. The input channels are represented by the second dimension, c .

3. Filters (or Kernels):

- Filters are the learnable weights applied to the images through convolution. A CNN often uses multiple filters in each layer to capture different patterns.
- Each filter slides across the image, computing a dot product between the filter weights and the image pixels. After applying a filter, the result is one output channel.
- If we have f filters, the output will have f channels, where each channel corresponds to the response of one filter.

Question:

Implement the function `conv_2d_batch`, following the provided instructions.

```
def conv_2d_batch(x, wt, b, p=0):
    """ A naive implementation of 2D convolution for a batch of images, and multiple filters.

    The input consists of N data points (images), each with C channels, height H and
    width W. We convolve each input with F different filters, where each filter
    spans all C channels and has height HH and width WW.

    Input:
    - x: a tensor of shape (n, c, h, w)
    - wt: a tensor of shape (f, c, hh, ww)
    - b: biases, of shape (f,)
    - p: an integer padding size

    Returns:
    - out: a tensor of shape (n, f, h', w') where
        h' = h + 2 * p - hh + 1
        w' = w + 2 * p - ww + 1
    """

    # Ensure all inputs are on the same device
    device = x.device
    wt = wt.to(device)
    b = b.to(device)

    # Get input dimensions
    n, c, h, w = x.shape
    # Get filter dimensions
    f, _, hh, ww = wt.shape # f is number of filters, hh and ww are filter height and width

    # Calculate output dimensions
    h_out = h + 2 * p - hh + 1
    w_out = w + 2 * p - ww + 1

    # Initialize output tensor
    out = torch.zeros((n, f, h_out, w_out), dtype=x.dtype, device=x.device)

    # Pad the input tensor
    # F.pad for 4D tensor (N, C, H, W) expects (pad_left, pad_right, pad_top, pad_bottom)
    if p > 0:
        x_padded = F.pad(x, (p, p, p, p), mode='constant')
    else:
        x_padded = x

    # Perform convolution
    for idx_n in range(n): # Loop over each image in the batch
        for idx_f in range(f): # Loop over each filter
            for i in range(h_out): # Loop over output height
                for j in range(w_out): # Loop over output width
                    # Extract the slice of the padded input for this position
                    # and sum over all input channels, then multiply by filter weights
                    # and add bias.
                    out[idx_n, idx_f, i, j] = torch.sum(
                        x_padded[idx_n, :, i:i+hh, j:j+ww] * wt[idx_f, :, :, :]
                    ) + b[idx_f]

    return out
```

The code below help you check the correctness of your implementation.

```
n = 2
c = 3
h = 4
w = 4
f = 3
hh = 3
ww = 3

x_shape = (n, c, h, w)
wt_shape = (f, c, hh, ww)
b_shape = (f,)

requires_grad = False

x = torch.rand(x_shape, dtype=torch.float32, requires_grad=requires_grad)
```

```

wt = torch.rand(wt_shape, dtype=torch.float32, requires_grad=requires_grad)
b = torch.rand(b_shape, dtype=torch.float32, requires_grad=requires_grad)
p = 1

out = conv_2d_batch(x, wt, b, p)

out_torch = F.conv2d(x, wt, bias=b, stride=1, padding=p)

result = torch.allclose(out, out_torch, atol=1e-4)
print("Is the result correct? ", result)

```

Is the result correct? True

▼ Part 2.2: Data Preparation

The FashionMNIST dataset is a popular dataset for image classification tasks. It contains 60,000 grayscale images in 10 classes in training, and 10,000 for testing. Each image is of size 28x28 pixels and has 1 color channels (grayscale). We'll divide this dataset into three sets:

- Training set: 48,000 images for training the model.
- Validation set: 12,000 images to evaluate the model's performance during training.
- Test set: 10,000 images to evaluate the final model.

First, we load the FashionMNIST dataset. This might take a couple minutes the first time you do it, but the files should stay cached after that.

```

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader
from torch.utils.data import sampler

import torchvision.datasets as dset
import torchvision.transforms as T
import torchvision.transforms as transforms

import numpy as np

```

```

NUM_TRAIN = 48000

# The torchvision.transforms package provides tools for preprocessing data
# and for performing data augmentation; here we set up a transform to
# preprocess the data by subtracting the mean RGB value and dividing by the
# standard deviation of each RGB value; we've hardcoded the mean and std.
transform = transforms.Compose(
    [transforms.ToTensor(),
     transforms.Normalize((0.5,), (0.5,))])

# We set up a Dataset object for each split (train / val / test); Datasets load
# training examples one at a time, so we wrap each Dataset in a DataLoader which
# iterates through the Dataset and forms minibatches. We divide the FashionMNIST
# training set into train and val sets by passing a Sampler object to the
# DataLoader telling how it should sample from the underlying Dataset.
fashion_mnist_train = dset.FashionMNIST('./datasets/fashion_mnist', train=True, download=True,
                                         transform=transform)
loader_train = DataLoader(fashion_mnist_train, batch_size=64,
                          sampler=sampler.SubsetRandomSampler(range(NUM_TRAIN)))

fashion_mnist_val = dset.FashionMNIST('./datasets/fashion_mnist', train=True, download=True,
                                       transform=transform)
loader_val = DataLoader(fashion_mnist_val, batch_size=64,
                       sampler=sampler.SubsetRandomSampler(range(NUM_TRAIN, 60000)))

fashion_mnist_test = dset.FashionMNIST('./datasets/fashion_mnist', train=False, download=True,
                                        transform=transform)
loader_test = DataLoader(fashion_mnist_test, batch_size=64)

```

```

100%|██████████| 26.4M/26.4M [00:02<00:00, 11.6MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 212kB/s]
100%|██████████| 4.42M/4.42M [00:01<00:00, 3.94MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 13.0MB/s]

```

Mini Data Loaders for Faster Testing:

To quickly test the model and ensure code correctness, we create mini versions of the training and validation loaders, which contain only a few samples. These mini loaders use a batch size of 4 and sample from a very small subset of the FashionMNIST dataset.

```
loader_train_mini = DataLoader(fashion_mnist_train, batch_size=4,
                              sampler=sampler.SubsetRandomSampler(range(4)))

loader_val_mini = DataLoader(fashion_mnist_val, batch_size=4,
                             sampler=sampler.SubsetRandomSampler(range(4, 6)))
```

Setting the Device for Computation:

Finally, we set the computation device to GPU if available, as this will significantly speed up training.

```
USE_GPU = True

dtype = torch.float32 # we will be using float throughout this tutorial

if USE_GPU and torch.cuda.is_available():
    device = torch.device('cuda')
else:
    device = torch.device('cpu')

print('using device:', device)

using device: cuda
```

By completing Part 2.2, you've now set up the FashionMNIST dataset and prepared the data loaders for efficient training and evaluation. In the next part, you'll use this dataset to train a custom CNN model built with the convolution functions you implemented.

✓ Part 2.3 Implementing the Custom CNN

In this part, you will implement a Convolutional Neural Network (CNN) model using the custom 2D convolution function you created in Part 1. This CNN will then be used to classify images in the FashionMNIST dataset. The structure of this model includes custom convolutional layers, ReLU activations, max-pooling layers, and fully connected (linear) layers to perform the final classification.

Custom Convolutional Layer:

Each convolutional layer in our network uses the custom convolution function (`conv_2d_batch`) that you implemented earlier. These custom layers replace the standard PyTorch convolutional layers to give you insight into how convolution works at a low level.

You are provided with the code for `CustomConv2DLayer`, which performs a 2D convolution operation using a custom function (`conv_2d_batch`). This layer initializes learnable weights and biases and uses them to compute the convolution.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from tqdm import tqdm

# Custom convolutional layer using our `conv_2d_batch` function
class CustomConv2DLayer(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size, padding=0):
        super(CustomConv2DLayer, self).__init__()
        self.padding = padding
        self.weight = nn.Parameter(torch.randn(out_channels, in_channels, kernel_size, kernel_size))
        self.bias = nn.Parameter(torch.randn(out_channels))

    def forward(self, x):
        # Use the custom conv_2d_batch function
        return conv_2d_batch(x, self.weight, self.bias, self.padding)
```

Model Structure: CustomCNN

The CustomCNN class represents the CNN model with multiple custom convolutional layers. Each convolutional layer is followed by a ReLU activation and a max-pooling layer for downsampling. Finally, the model has fully connected layers for classification.

You will implement a simple CNN architecture with the following general structure:

- Conv Layer 1: A custom convolutional layer with 1 input channels and 16 output channels, kernel size of 3, and padding of 1.
- Activation Function: Use the ReLU activation function after the first convolutional layer.
- Pooling: Apply max pooling with a kernel size of 2 to reduce the spatial dimensions by half.
- Conv Layer 2: A second custom convolutional layer, with 16 input channels and 32 output channels, kernel size of 3, and padding of 1.
- Activation Function: Apply ReLU again after this layer.
- Pooling: Use max pooling with a kernel size of 2 to reduce the dimensions further.
- After the convolutional layers, flatten the data into a 1D vector to feed into fully connected layers.
- Add a fully connected layer with 128 units, followed by a ReLU activation function.
- Add a final fully connected layer with 10 units (one for each FashionMNIST class).

Hints and Tips:

- Use `F.relu()` for the ReLU activation.
- To downsample the spatial dimensions, use `F.max_pool2d()` with kernel size 2 and stride 2.
- Flatten the tensor using `x = x.view(x.size(0), -1)` before passing it to the fully connected layers.
- Ensure the output layer has the correct number of units (10 for FashionMNIST classes).

```
# Define a CNN model with multiple custom convolutional layers
class CustomCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(CustomCNN, self).__init__()
        # Conv Layer 1
        self.conv1 = CustomConv2DLayer(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        # Conv Layer 2
        self.conv2 = CustomConv2DLayer(in_channels=16, out_channels=32, kernel_size=3, padding=1)

        # Calculate flattened size: input size 28x28
        # After conv1 (16, 28, 28) -> max_pool (16, 14, 14)
        # After conv2 (32, 14, 14) -> max_pool (32, 7, 7)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        # Implement the forward pass in the forward method, where you pass data
        # sequentially through each layer, applying activations and pooling as
        # specified.

        # Conv Layer 1 -> ReLU -> Max Pooling
        x = self.conv1(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2, stride=2)

        # Conv Layer 2 -> ReLU -> Max Pooling
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2, stride=2)

        # Flatten the data
        x = x.view(x.size(0), -1)

        # Fully Connected Layers
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)

        return x
```

Once implemented, test your model with the synthetic data, to ensure it runs without errors. To do this, define a dummy input `x = torch.randn(4, 1, 28, 28)`, and print the forward pass of our model using `model(x)`

```
model = CustomCNN(num_classes=10).to(device)
x = torch.randn(4, 1, 28, 28, dtype=torch.float32, device=device)
output = model(x)
print("Output shape:", output.shape)
print("Output:", output)
```

```
Output shape: torch.Size([4, 10])
Output: tensor([[ 0.6004, -1.6060, 10.0564,  3.1034,  7.5973,  2.9396, -5.0623,
```

```

5.3104, -3.2232, -11.8555],
[ -0.1030, -3.2143, 10.8995, 1.7179, 7.6787, 7.1051, -0.3043,
1.9293, -1.5655, -8.8363],
[ 1.9064, 1.5313, 9.7521, -0.5620, 5.5112, 7.7113, -3.4261,
4.5224, -0.6917, -8.4024],
[ 2.9041, -0.5321, 11.0574, 3.9643, 4.8538, 4.6329, -1.6409,
4.0730, -1.3165, -8.9013]], device='cuda:0',
grad_fn=<AddmmBackward0>)

```

Part 2.4: Implementing the Training Function and Testing on a Mini Dataset

In this part, you will implement a training function to train `CustomCNN` on a mini dataset of FashionMNIST. Training on a small sample will help verify that the model and training loop work as expected before running on the full dataset.

The training function should perform the following steps:

- Set the model to training mode
- Loop over the training data for each epoch
- For each batch, move the data and labels to the appropriate device (CPU or GPU)
- Forward pass: Use the model to make predictions
- Compute the loss
- Backward pass: Calculate the gradient for the parameters Update the model's parameters using an optimizer.
- Print training metrics such as loss every few iterations to monitor progress.

You can use the code from lab 9 for reference. Insert code below.

```

def check_accuracy(loader, model, device):
    if loader.dataset.transform is not None:
        print('Checking accuracy on validation set')
    else:
        print('Checking accuracy on test set')
    num_correct = 0
    num_samples = 0
    model.eval() # set model to evaluation mode
    with torch.no_grad():
        for x, y in loader:
            x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
            y = y.to(device=device, dtype=torch.long)
            scores = model(x)
            _, preds = scores.max(1)
            num_correct += (preds == y).sum()
            num_samples += preds.size(0)
    acc = float(num_correct) / num_samples
    print('Got %d / %d correct (%.2f)' % (num_correct, num_samples, 100 * acc))
    return acc

def train(model, optimizer, epochs=1, loader_train=None, loader_val=None, device=None):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(epochs):
        model.train() # set model to training mode
        running_loss = 0.0
        for t, (x, y) in enumerate(loader_train):
            x = x.to(device=device, dtype=dtype) # move to device, e.g. GPU
            y = y.to(device=device, dtype=torch.long) # move to device

            scores = model(x)
            loss = criterion(scores, y)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            if t % 100 == 0:
                print('Epoch %d, Iteration %d, loss = %.4f' % (epoch, t, loss.item()))

        print('Epoch %d, finished with average loss = %.4f' % (epoch, running_loss / len(loader_train)))
        if loader_val is not None:
            check_accuracy(loader_val, model, device)

    return model

```


Once implemented, we create a model, and train the model using the mini dataset `loader_train_mini` and `loader_val_mini`.

Train the model on `loader_train_mini` for a few epochs (e.g. 5-10 epochs) and observe the printed loss and accuracy. This helps to ensure the function is error-free.

Since you are training on a very small dataset, you may see fluctuations in loss and accuracy. However, the purpose here is to ensure there are no errors in the training function or model implementation.

To measure the time spent on training, we can use Python's `time` module. Here's a function that you can add to measure the time taken by training.

```
import time

# Your code to train the model..
model = CustomCNN(num_classes=10)
model.to(device)

start_time = time.time() # Record the start time

### Your training code ###
optimizer = optim.Adam(model.parameters(), lr=1e-3) # Define optimizer

train(
    model,
    optimizer,
    epochs=5, # Train for 5 epochs on the mini dataset
    loader_train=loader_train_mini,
    loader_val=loader_val_mini,
    device=device
)

end_time = time.time() # Record the end time
elapsed_time = end_time - start_time # Calculate the elapsed time

print(f"Training took {elapsed_time:.2f} seconds.")
```

```
Epoch 0, Iteration 0, loss = 4.1957
Epoch 0, finished with average loss = 4.1957
Checking accuracy on validation set
Got 0 / 2 correct (0.00)
Epoch 1, Iteration 0, loss = 17.9367
Epoch 1, finished with average loss = 17.9367
Checking accuracy on validation set
Got 0 / 2 correct (0.00)
Epoch 2, Iteration 0, loss = 13.3664
Epoch 2, finished with average loss = 13.3664
Checking accuracy on validation set
Got 1 / 2 correct (50.00)
Epoch 3, Iteration 0, loss = 0.9098
Epoch 3, finished with average loss = 0.9098
Checking accuracy on validation set
Got 1 / 2 correct (50.00)
Epoch 4, Iteration 0, loss = 1.7828
Epoch 4, finished with average loss = 1.7828
Checking accuracy on validation set
Got 1 / 2 correct (50.00)
Training took 131.37 seconds.
```

✓ Part 2.5: Replacing Custom Convolutional Layers with `nn.Conv2d`

In this part, you'll replace the custom convolutional layers you implemented with PyTorch's built-in `nn.Conv2d()` function. By doing this, you'll be able to observe the difference in performance between your own implementation and the optimized `nn.Conv2d()` layers. This comparison will highlight the efficiency of using well-optimized built-in layers in real-world applications.

Instructions:

1. Create a New CNN Model Using `nn.Conv2d`:

- Replace `CustomConv2DLayer` with `nn.Conv2d` in your `CustomCNN` class.
- Make sure the layer configurations (input channels, output channels, kernel size, and padding) match your previous custom model.
- Keep the fully connected layers and the rest of the structure the same as before.

2. Train on the Mini Dataset:

- Use the same mini dataset you created earlier to train your new model with `nn.Conv2d`.
- Record the training time for this model.

3. Compare Training Times:

- After training, compare the time taken by your model with `nn.Conv2d` to the time taken by your custom `CustomConv2DLayer` implementation.
- Reflect on why `nn.Conv2d` is faster—consider how libraries like PyTorch optimize operations for efficiency, especially on hardware like GPUs.

Insert your code cells below.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class FastCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(FastCNN, self).__init__()
        # Conv Layer 1: using nn.Conv2d
        self.conv1 = nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, padding=1)
        # Conv Layer 2: using nn.Conv2d
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)

        # Calculate flattened size: input size 28x28
        # After conv1 (16, 28, 28) -> max_pool (16, 14, 14)
        # After conv2 (32, 14, 14) -> max_pool (32, 7, 7)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, num_classes)

    def forward(self, x):
        # Conv Layer 1 -> ReLU -> Max Pooling
        x = self.conv1(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2, stride=2)

        # Conv Layer 2 -> ReLU -> Max Pooling
        x = self.conv2(x)
        x = F.relu(x)
        x = F.max_pool2d(x, kernel_size=2, stride=2)

        # Flatten the data
        x = x.view(x.size(0), -1)

        # Fully Connected Layers
        x = self.fc1(x)
        x = F.relu(x)
        x = self.fc2(x)

        return x
```

Part 2.5: Train and Evaluate FastCNN on the FashionMNIST Dataset

Now that you have timed the training process for FastCNN on a mini-dataset. If no error occurs, let's use the full FashionMNIST dataset to evaluate your optimized FastCNN model with `nn.Conv2d` layers.

1. Train the FastCNN model using the full training dataset (`loader_train`).
2. Evaluate its performance on the validation set (`loader_val`) after each epoch.
3. Observe the accuracy and loss trends to see how well your model performs on FashionMNIST.

Insert your code cells below.

```
import time

# 2. Train on the Mini Dataset:
print("Training FastCNN on mini dataset...")
model_fast_mini = FastCNN(num_classes=10).to(device)
optimizer_fast_mini = optim.Adam(model_fast_mini.parameters(), lr=1e-3)

start_time_fast_mini = time.time() # Record the start time
```

```
train(
    model_fast_mini,
    optimizer_fast_mini,
    epochs=5, # Train for 5 epochs on the mini dataset
    loader_train=loader_train_mini,
    loader_val=loader_val_mini,
    device=device
)

end_time_fast_mini = time.time() # Record the end time
elapsed_time_fast_mini = end_time_fast_mini - start_time_fast_mini # Calculate the elapsed time

print(f"FastCNN (mini) training took {elapsed_time_fast_mini:.2f} seconds.\n")

# 3. Train and Evaluate on the Full FashionMNIST Dataset
print("Training FastCNN on full FashionMNIST dataset...")
model_fast = FastCNN(num_classes=10).to(device)
optimizer_fast = optim.Adam(model_fast.parameters(), lr=1e-3)

train(
    model_fast,
    optimizer_fast,
    epochs=10, # You can adjust the number of epochs
    loader_train=loader_train,
    loader_val=loader_val,
    device=device
)

print("FastCNN training on full dataset complete.")
```

```
Epoch 9, Iteration 500, loss = 0.1834
Epoch 9, Iteration 600, loss = 0.1989
Epoch 9, Iteration 700, loss = 0.1284
Epoch 9, finished with average loss = 0.1420
Checking accuracy on validation set
Got 11038 / 12000 correct (91.98)
FastCNN training on full dataset complete.
```

Part 3: Testing Your Final Model on the FashionMNIST Test Set (Run this only once!)

After you've completed your experiments and achieved a satisfactory validation accuracy, the final step is to evaluate your model on the test set. This will give you an objective measure of how well your model generalizes to unseen data.

Question:

Test your model on the test dataset `loader_test`.

What is the accuracy of your model on the test set?

How is it compared to your validation accuracy?

```
print("Checking accuracy on test set...")
check_accuracy(loader_test, model_fast, device)
```

```
Checking accuracy on test set...
Checking accuracy on validation set
Got 9123 / 10000 correct (91.23)
0.9123
```

ANSWER: The accuracy is 91.23% which is very close, albeit slightly slower than, the validation set accuracy.

Lab 10 is now complete. Make sure all cells are visible and have been run (rerun if necessary).

The code below converts the ipynb file to PDF, and saves it to where this .ipynb file is.

```
NOTEBOOK_PATH = # Enter here, the path to your notebook file, e.g. "/content/drive/MyDrive/ECEN250/ECEN250_Lab10.ipynb". Do not
! pip install -U nbconvert playwright
! playwright install-deps
! jupyter nbconvert --to webpdf --allow-chromium-download "$NOTEBOOK_PATH"
```

Download your notebook as an .ipynb file, then upload it along with the PDF file (saved in the same Google Drive folder as this notebook) to Canvas for Lab 10. Make sure that the PDF file matches your .ipynb file.